

# From Zero to Multi-Device Assays

🇩🇰 Copenhagen · 28 September 2026

```
[1]: import pylabrobot_workshop  
pylabrobot_workshop.setup()
```

## TRAINERS

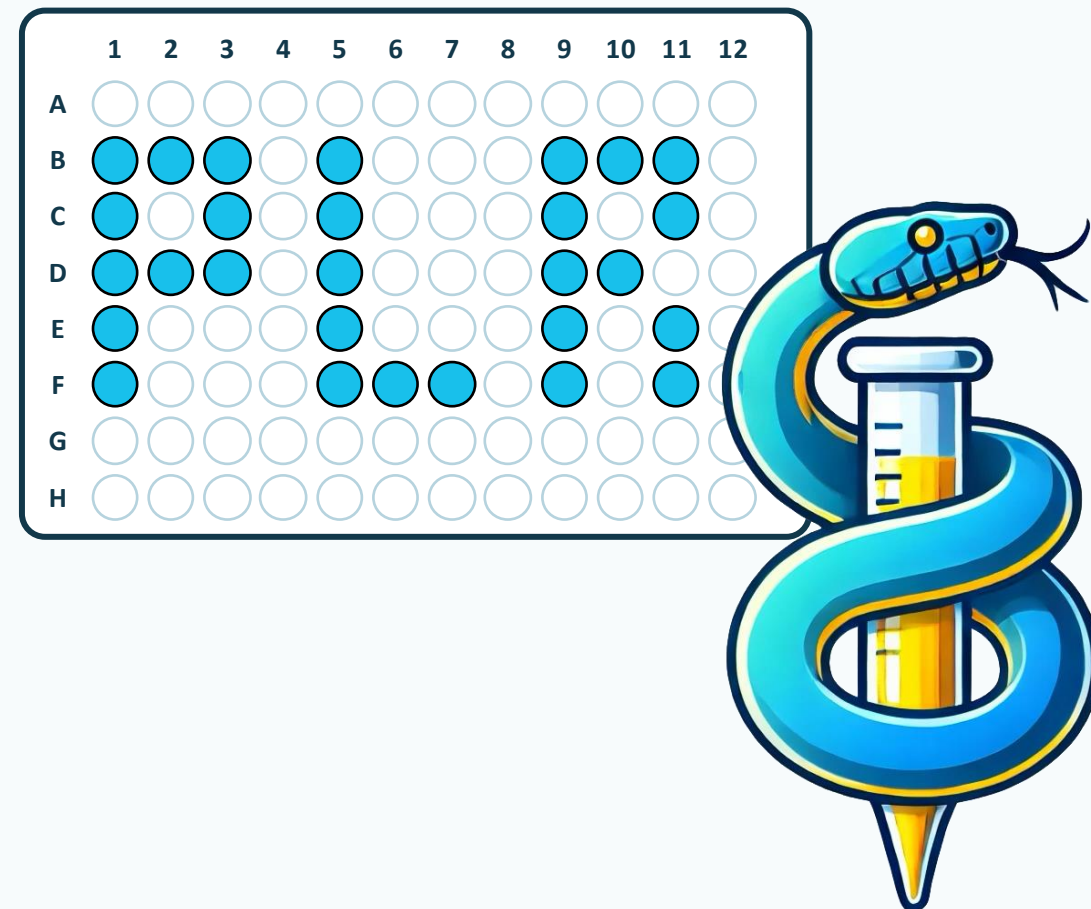
Rick Wierenga, Camillo Moschner

## ORGANISER

Kim Fris Olsson

## SPONSORS

Hamilton Company · Opentrons · Byonoy





# Intro to PyLabRobot

**Session 1** 09:00 – 10:00 · who we are, why program your automation, and how PLR is built



▶ hands-on in a notebook

# Your trainers: Rick Wierenga



Universiteit  
Leiden

BSc Bioinformatics + AI

+



Visiting Student in Esvelt Lab

(Building PyLabRobot)

Retro  
BIOSCIENCES

# Your trainers: Camillo Moschner, PhD



**BSc Biomedical  
Sciences & Molecular  
Biology (First Class)**



**UNIVERSITY OF  
CAMBRIDGE**

**PhD Engineering**

**Technology development for high-  
throughput genetic engineering &  
microfluidics-enabled microscopy  
studies**



**T-Therapeutics**

**Scientist**

**Built novel genetic  
engineering workflows &  
automated them**



**ConstructiveBio**

**Senior Scientist**

**Heading all things  
automation**



**Undergrad  
intern**



**2017  
2022**

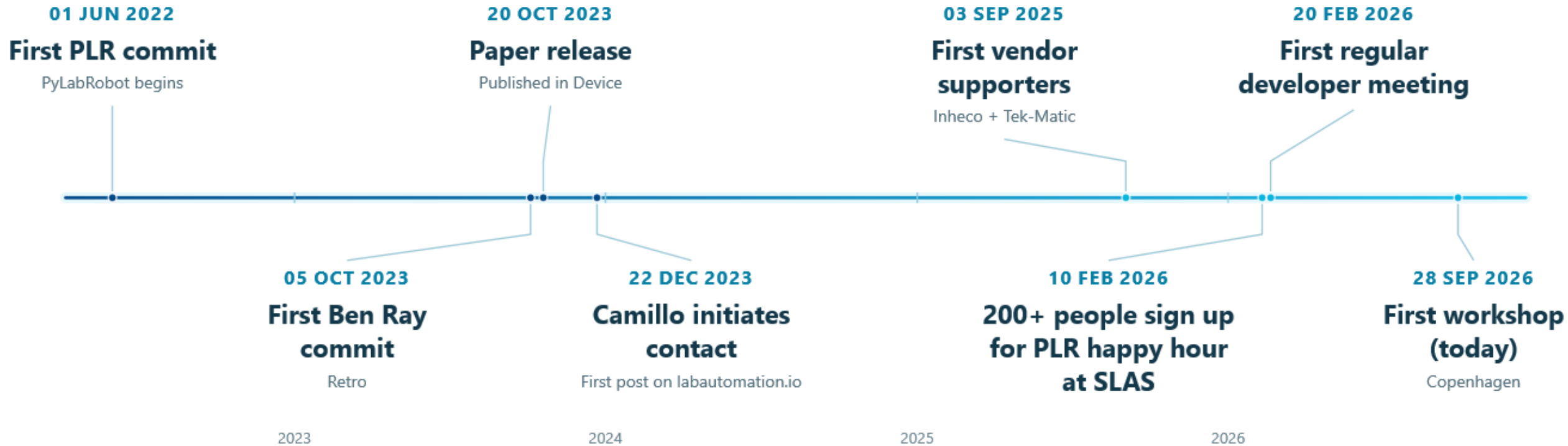


**PhD intern  
NGS automation**

**PLR developer since December 2023  
#2, ~430 commits, 388,683 ++ 247,764 –  
device manger: Hamilton STAR, CLARIOstar**



# A Short History of PLR





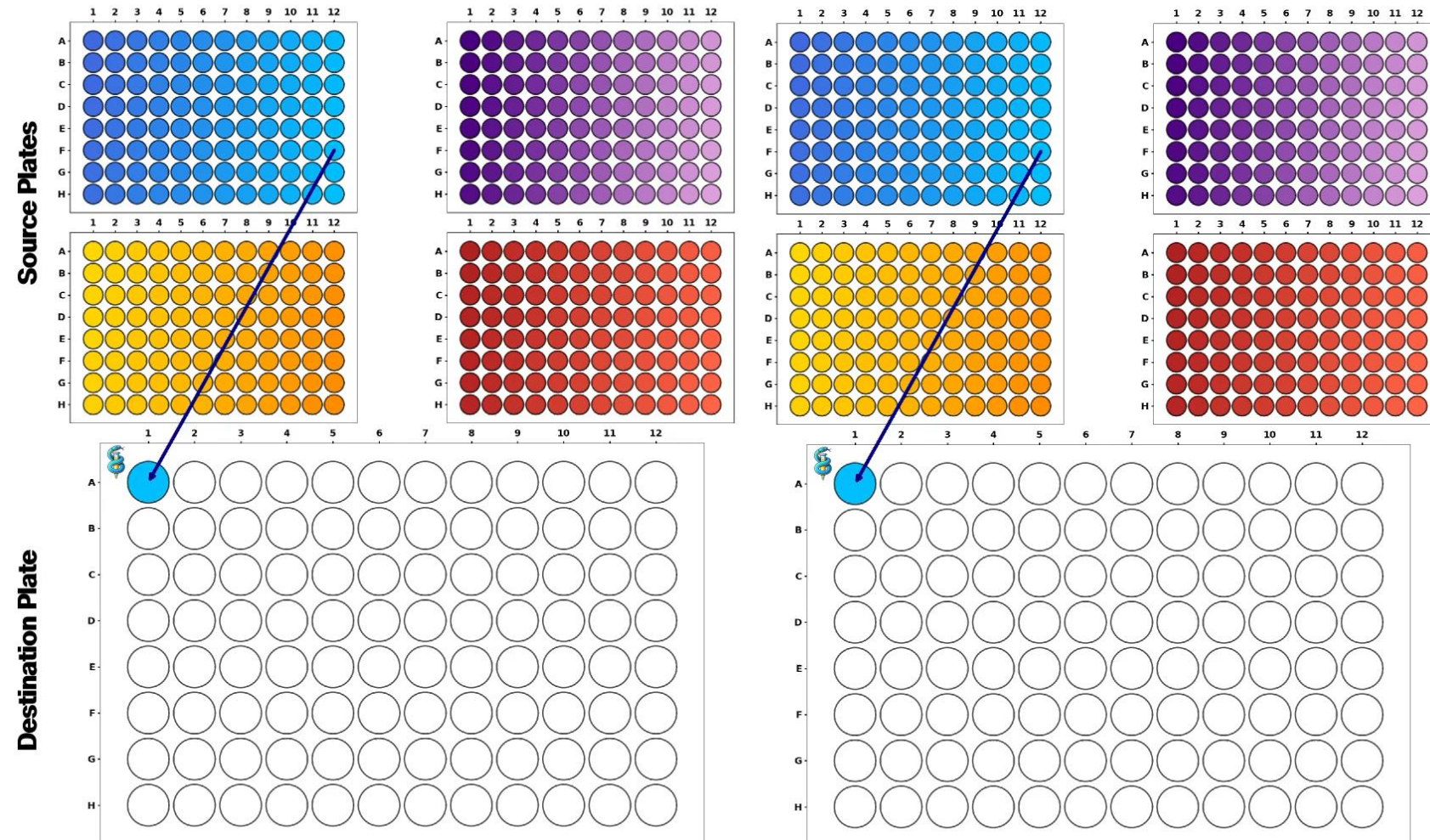
# A Short History of PLR – Camillo



- Built in-house cloning workflows
- Required large part libraries
- Hit identification  
-> cherry picking from libraries

- ➔ click-and-drag solutions required too many unstable hacks
- ➔ Needed a solution... PyLabRobot

## Visualisation of Complex Cherry Picking



# Overview



## SESSION 1 09:00 – 10:00

### Intro to PLR

- Resource Management System (“Digital Twin”)
- Device Management System (“Drivers”)



Coffee  
10:00  
15 min

## SESSION 2 10:15 – 12:00

### Probing & Verification Systems

#### Case study I: Normalisation

- Simple liquid handling to managed application

Lunch 12:00 – 13:00

## SESSION 3 13:00 – 14:30

### Case study II: Hit the Colour

- Using liquid handler & plate reader for end-to-end workflow



Coffee  
14:30  
15 min

## SESSION 4 14:45 – 16:00

### End-user Delivery Options

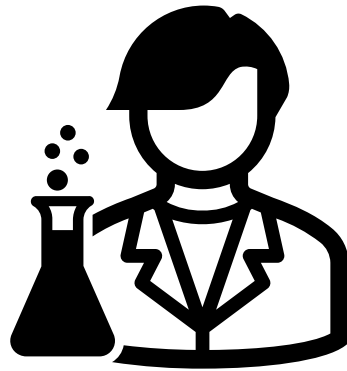
#### Plans & Future

- (time dependent)



# A Tale of Two Automators

Jens

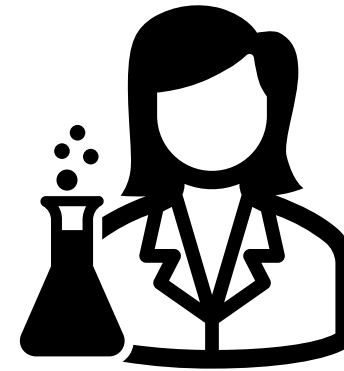


Click-and-drag  
automator



MONDAY

Normalise 96 samples  
by Friday



Inge

Programmatic  
automator



# A Tale of Two Automators



**Click-and-drag  
automator**



**MONDAY**  
Normalise 96 samples  
by Friday



**Programmatic  
automator**

**TUESDAY** First dry run: both protocols run in the simulator ✓

*Real problems: click a warning sign for its forum thread*

Resets the tip counter by hand.



**WEDNESDAY**  
Tip usage changed

Re-teaches the offsets.



**WEDNESDAY**  
Plate type changed

One setting for all channels.



**THURSDAY**  
Low sample volumes

Second program, export, scheduler.



**THURSDAY**  
Reader-decision needed

One editor at a time.



**FRIDAY**  
A colleague joins

Rebuilds every labware definition.



**NEXT MONTH**  
Liquid handler changed

Asks the model for tips.

Probes the plate.

Each channel probes own height,  
computes volume & adapts.

**import** scipy, retry\_helper

Pull request, review, merge.

Same resource definitions.

# Four Pillars of Programmatic Automation



ROBUST · PRECISE · INTEGRATED · SELF-DRIVING

PyLabRobot · Device Management System (drivers)

M

## MEASURE

*Don't assume. Measure.*

### State-verification

Verify tips, plates, carriers and liquid are there, before run comits.

### Auto-Calibration

Let robot measure their own positions and geometry.

C

## CONTROL

*Command every channel.*

### Decide per channel, not per step

Each channel uses its own parameters, at runtime.

### Dynamic scaling (is free)

8 transfers or 732: the same effort.

### Choose abstraction level

From one-line transfers to single firmware commands.

Python · any language

H

## HARNESS

*Access the world's prior work.*

### Import solutions

pandas, scipy, pytorch ... any tool

### Find edge cases before they find you

Generate hundreds of inputs; simulate them all.

### AI as a teammate

Generate, verify, brainstorm.

PyLabRobot · open source

O

## OWN

*Own your stack.*

### Write once, run on any robot

The same resource definitions on the Prep and the STAR.

### Fix and extend it yourself

Build what your lab needs, instead of waiting for it.

### Keep old hardware alive

Give valued instruments a longer working life.

PyLabRobot · Resource Management System

**RESOURCE MODEL** A living twin: updated by every measurement and action, asked before every decision.

Distinct Principles

# Overview



## SESSION 1 09:00 – 10:00

### RESOURCE MODEL

#### Intro to PLR

- Resource Management System (“Digital Twin”)
- Device Management System (“Drivers”)



Coffee  
10:00  
15 min

## SESSION 2 10:15 – 12:00



### Probing & Verification Systems

#### Case study I: Normalisation

- Simple liquid handling to managed application

## Lunch 12:00 – 13:00

## SESSION 3 13:00 – 14:30



#### Case study II: Hit the Colour

- Using liquid handler & plate reader for end-to-end workflow



Coffee  
14:30  
15 min

## SESSION 4 14:45 – 16:00

### End-user Delivery Options

#### Plans & Future

- (time dependent)



# PLR Architecture

# PLR Architecture Basic



YOUR PROTOCOL Python: a script or a notebook

Resource Management System (RMS)

STAR()  
*is a Resource*

Device Management System (DMS)

The digital twin: what is where

Deck STARDeck

Carrier PLT\_CAR\_L5AC

Plate Cor\_96\_wellplate

Well A1

Static resources

X-arm

Pipette

Mounting shaft

Tip HamiltonTip

Dynamic resources

has a

The drivers: what the device can do

.driver

.x\_arm

.pipettes .aspirate()

.iswap .pick/drop\_up\_resource()

.head96 .pick\_up\_tips()

... and .autoload,  
.core\_grippers

pylabrobot.io USB · Serial · FTDI · HID · Socket · HTTP

HARDWARE liquid handlers, plate readers, grippers, ...

# nb: PLR Architecture Provides Power

---



**Notebook 1: Step-by-step build of our simulation-execution workcell**



# PLR Architecture Basics – RMS Terms



Modelling static resources (*would you call these labware?*)

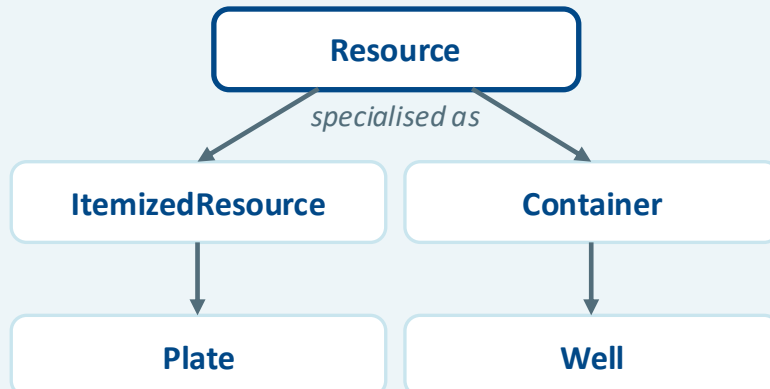
1

## Resource

**Everything is a cuboid:** a name, a size and a location in its parent.

`Resource(name, size_x, size_y, size_z)`

Subclasses specialise it and inherit the rest.



2

## (Coordinate) Frame

Positions are relative to the parent's origin: its left-front-bottom corner.

`Coordinate(x=10, y=20, z=0)`

3

## get\_commands

Ask the tree: where is it, how big, which one?

`get_absolute_location()`

`get_resource("plate")`

4

## assign\_ / unassign\_

Change the tree: place, remove; moving is unassign, then assign.

`assign_child_resource()`

`unassign_child_resource()`

5

## Deck

The layout of a device where resources are positioned at defined locations.

`STARDeck`

6

## Workcell

All equipment joined into one collaborating unit, in one shared frame.

7

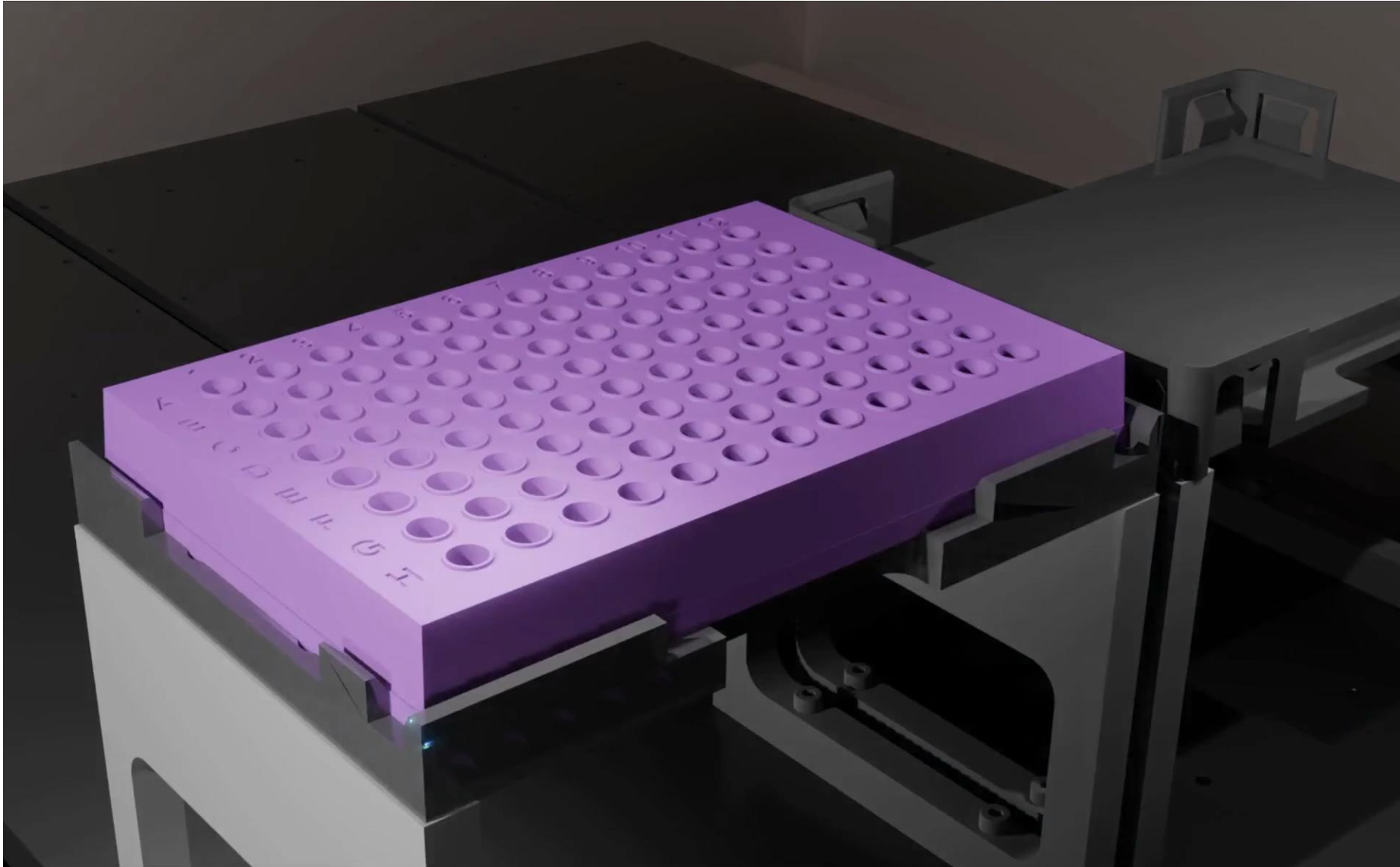
## Facility

The room: the root frame everything else resolves against.

`Facility()`

More terms in the glossary: [biocamtechnologies.com/automation/glossary/](https://biocamtechnologies.com/automation/glossary/)

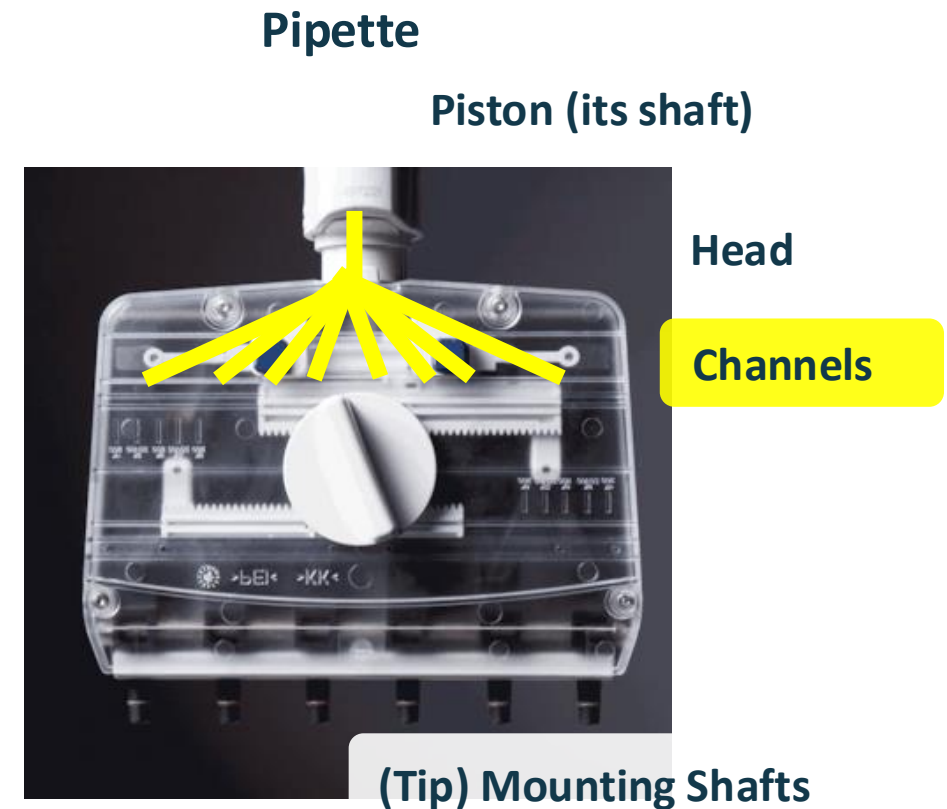
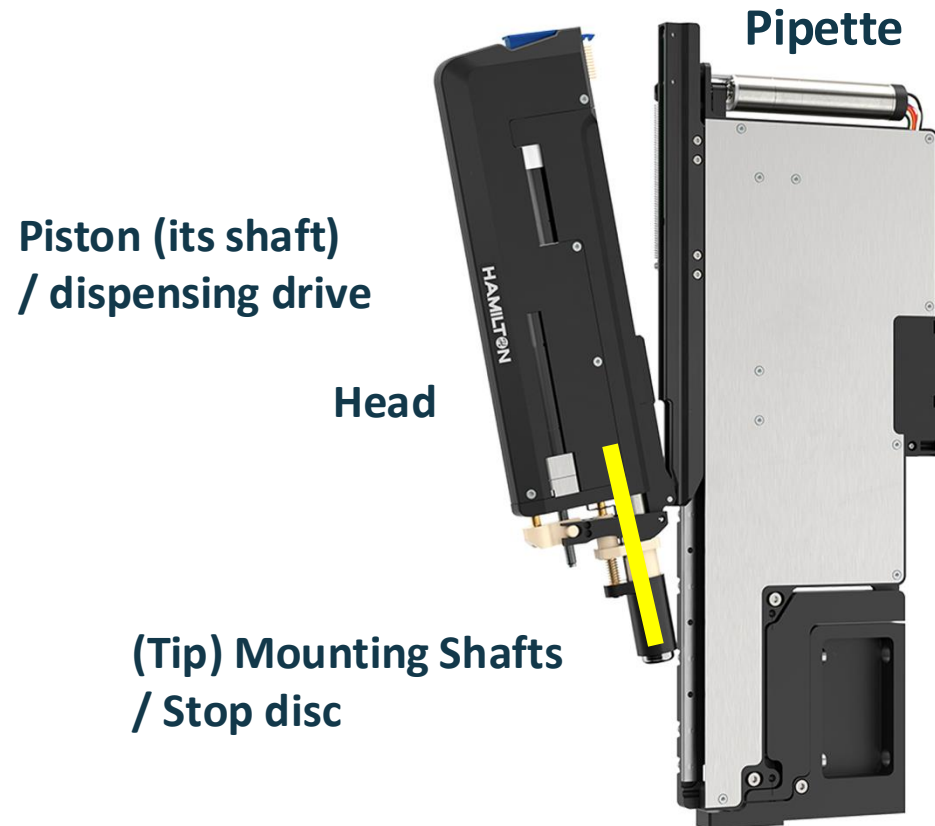
# PLR Architecture Basics – RMS Terms



# PLR Architecture Basics – RMS Terms



Modelling dynamic resources *(the parts of a device that move and act in space)*



More terms in the glossary: [biocamtechnologies.com/automation/glossary/](https://biocamtechnologies.com/automation/glossary/)

# PLR Architecture Basics – DMS Terms

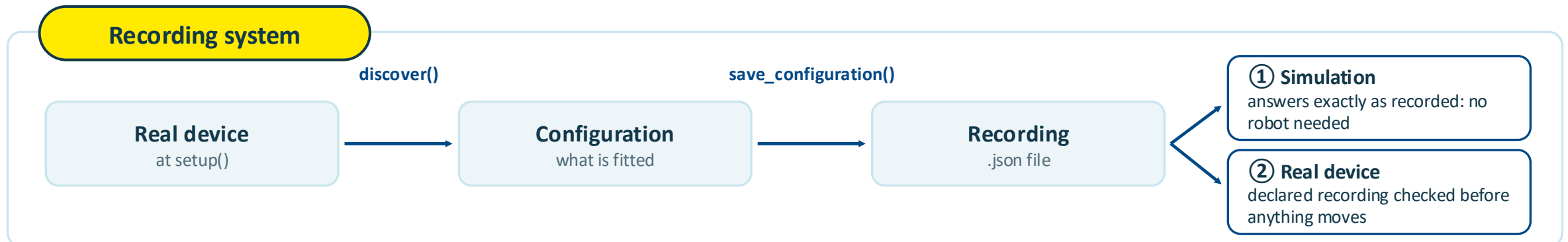


Driving the device *(the code that makes the parts move and act)*

```
prep PrepDevice
├── .deck
├── .driver PrepDriver | PrepSimulationDriver
├── .x_arm
├── .pipettes
├── .core_grippers
├── .head8
├── .lights
├── .method
└── .calibration
```

```
star STARDevice
├── .deck
├── .driver STARDriver | STARSimulationDriver
├── .x_arm
├── .pipettes
├── .core_grippers
├── .head96/384
├── .iswap
├── .autoload
└── .front_cover
```

# yellow: same on both devices



# **nb: PLR Architecture Provides Power**

---



## **Notebook 2: “Hamilton heaven” power showcase**

**What you can achieve very quickly with PLR’s resource management system.**



# Our Devices Today

# Workshop Devices



Hamilton Prep

PrepDevice



Execution

Simulation

Hamilton STAR

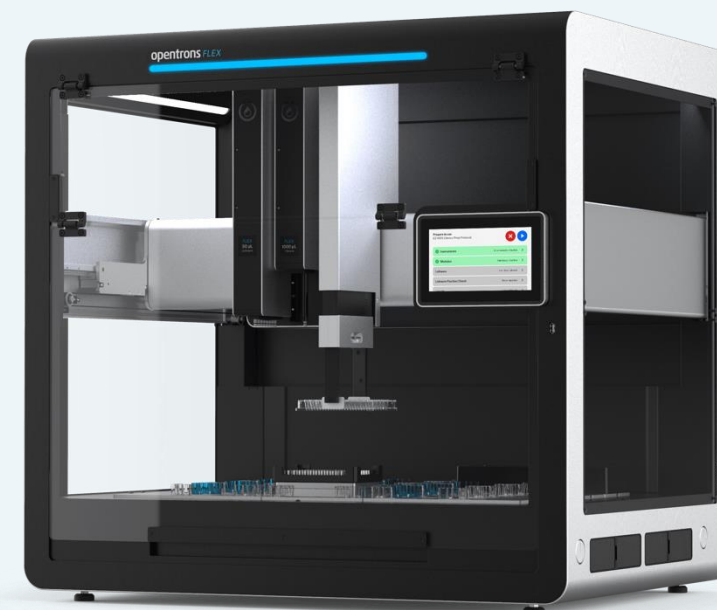
STARDevice



Simulation

Opentrons Flex

Flex



Execution

Simulation

# Workshop Devices



## Hamilton Prep

PrepDevice



|            |                                     |
|------------|-------------------------------------|
| Size       | Benchtop, 497 × 575 × 575 mm        |
| Pipetting  | 2 channels; optional 8-channel head |
| Grippers   | CoRe grippers                       |
| Loading    | by hand                             |
| Sensing    | capacitive + pressure LLD, zTouch   |
| Connection | Ethernet (TCP)                      |

## Hamilton STAR

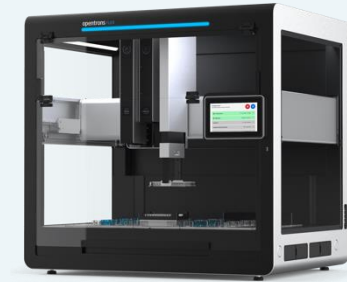
STARDevice



|            |                                   |
|------------|-----------------------------------|
| Size       | 1,667 mm wide, 54 tracks          |
| Pipetting  | 8 channels + 96-channel head      |
| Grippers   | iSWAP + CoRe grippers             |
| Loading    | autoload (carriers in and out)    |
| Sensing    | capacitive + pressure LLD, zTouch |
| Connection | USB                               |

## Opentrons Flex

Flex



|            |                                                    |
|------------|----------------------------------------------------|
| Size       | Benchtop, 12 slots + 4 staging                     |
| Pipetting  | two mounts (1 or 8 channels), or a 96-channel head |
| Grippers   | gripper                                            |
| Loading    | by hand                                            |
| Sensing    | liquid probe                                       |
| Connection | network (HTTP)                                     |



# Workshop Devices



## The same in PyLabRobot

### One resource model

Deck, carriers, plates, tips: the same Resource tree

### Same resource definitions

A plate defined once works on both

### Same shape of device

`.driver` `.x_arm` `.pipettes` `.core_grippers`

### Simulation from recordings

Both run without hardware first

## Different on the bench

### Scale

2 channels vs 8 channels + 96-head

### Deck

A few fixed positions vs 54 tracks of carriers

### Moving plates

CoRe grippers vs iSWAP + CoRe grippers

### Connection

Ethernet vs USB



# Probing & Verification

**Session 2** 10:15 – 12:00 · harness your device's sensors, then put them to work

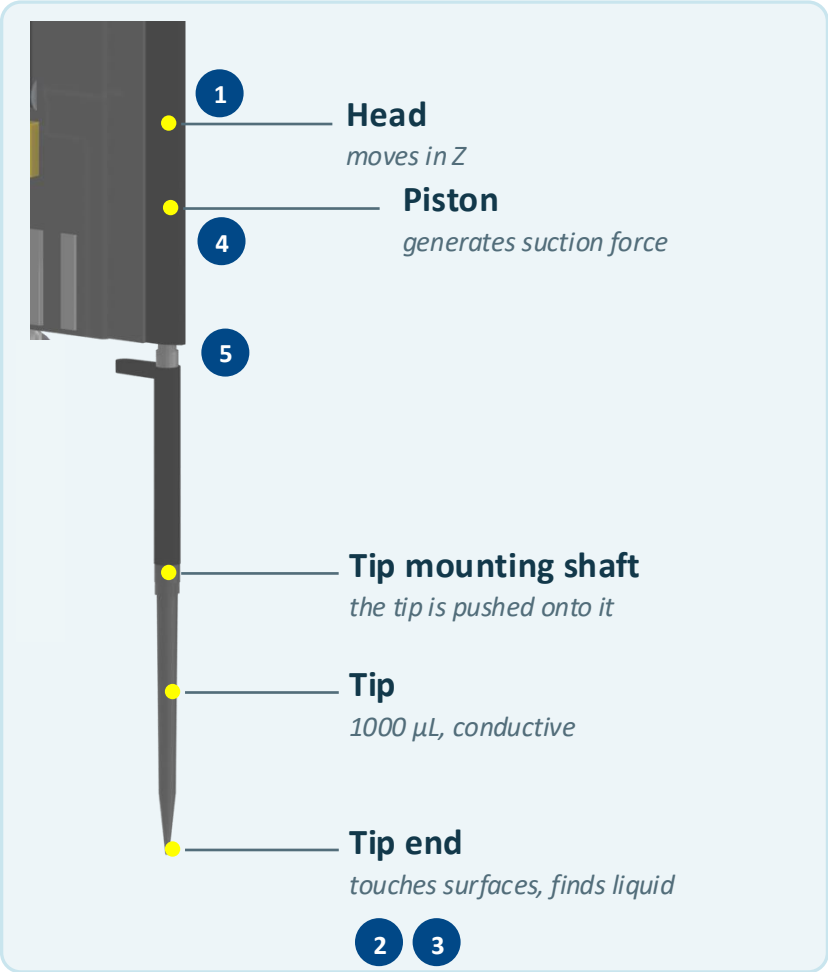


▶ hands-on in a notebook

# Your Device's Senses



Every channel is a sensor (*it can find things, not just move to them*)



| Sense            | Finds                                 | Method                                   | Prep                    | STAR |
|------------------|---------------------------------------|------------------------------------------|-------------------------|------|
| 1 zTouch (force) | any solid surface, in Z               | probe_z_using_ztouch                     | ✓                       | ✓    |
| 2 capacitive LLD | liquid or a conductive surface, in Z  | probe_z_using_clld                       | ✓                       | ✓    |
| 3 capacitive LLD | edges of conductive parts, in X and Y | probe_x_using_clld<br>probe_y_using_clld | ✓                       | ✓    |
| 4 pressure LLD   | the liquid surface, in Z              | probe_z_using_plld                       | in aspirate in aspirate |      |
| 5 tip sensor     | a tip on each channel                 | sense_tip_presence                       | ✓                       | ✓    |

# nb: Probing Demonstration

---



## Notebook 3: Probing on the Hamilton Prep

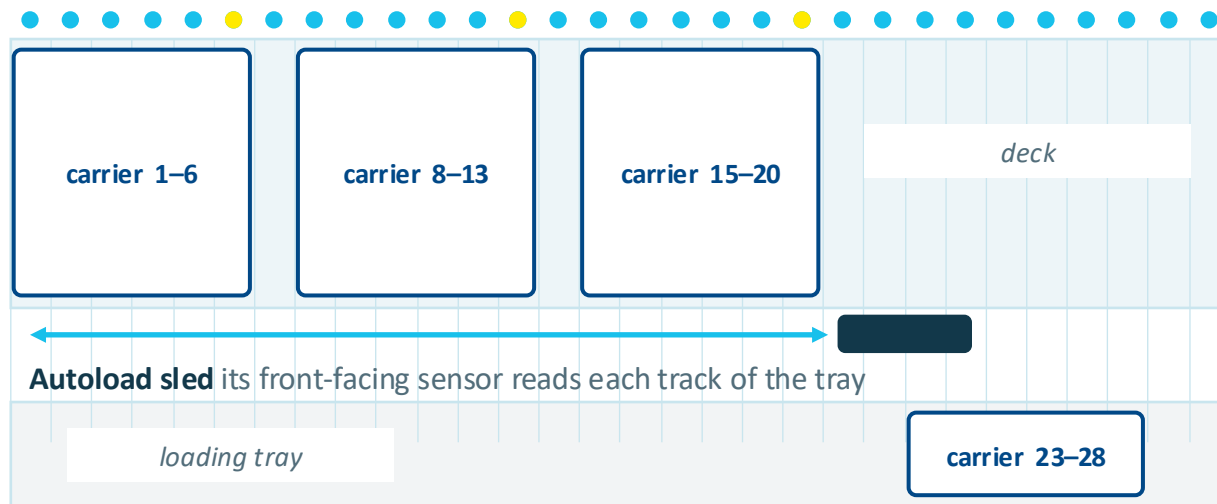
### Deck offset & tip presence

# Your Device's Senses – Beyond Pipettes



The STAR knows which tracks hold a carrier (*it can find things, not just move to them*)

Rear deck sensors one per track; **yellow**: a carrier's right-most track



```
# the deck vs. the model
found = await star.autoload
        .sense_carrier_presence_on_deck()
```

```
planned = {
    star.deck.compute_right_track_of_carrier(c)
    for c in carriers
}
```

```
missing = planned - found # not there
extra = found - planned # not modelled
```

1

## On the deck

Reads the rear sensors. The autoload does not move.  
`autoload.sense_carrier_presence_on_deck()`  
→ [6, 13, 20]

2

## On the loading tray

The sled drives across the tray and reads each track.  
`autoload.sense_carrier_presence_on_loading_tray()`  
→ [28]



3

4

5

## the tips



► Notebook: d1\_s2\_nb4\_simple\_lh

# nb: Simple Liquid Handling

---



## Notebook 4: Simple liquid handling

... but

- on 2 different physical devices &
- 1 simulated device

...at the same time 😊



# Case Study I: Normalisation

**Session 2** 10:15 – 12:00 · harness your device's sensors, then put them to work



▶ hands-on in a notebook



# Case study I: Normalisation



Problem we are solving: every sample at its own concentration, one target for all

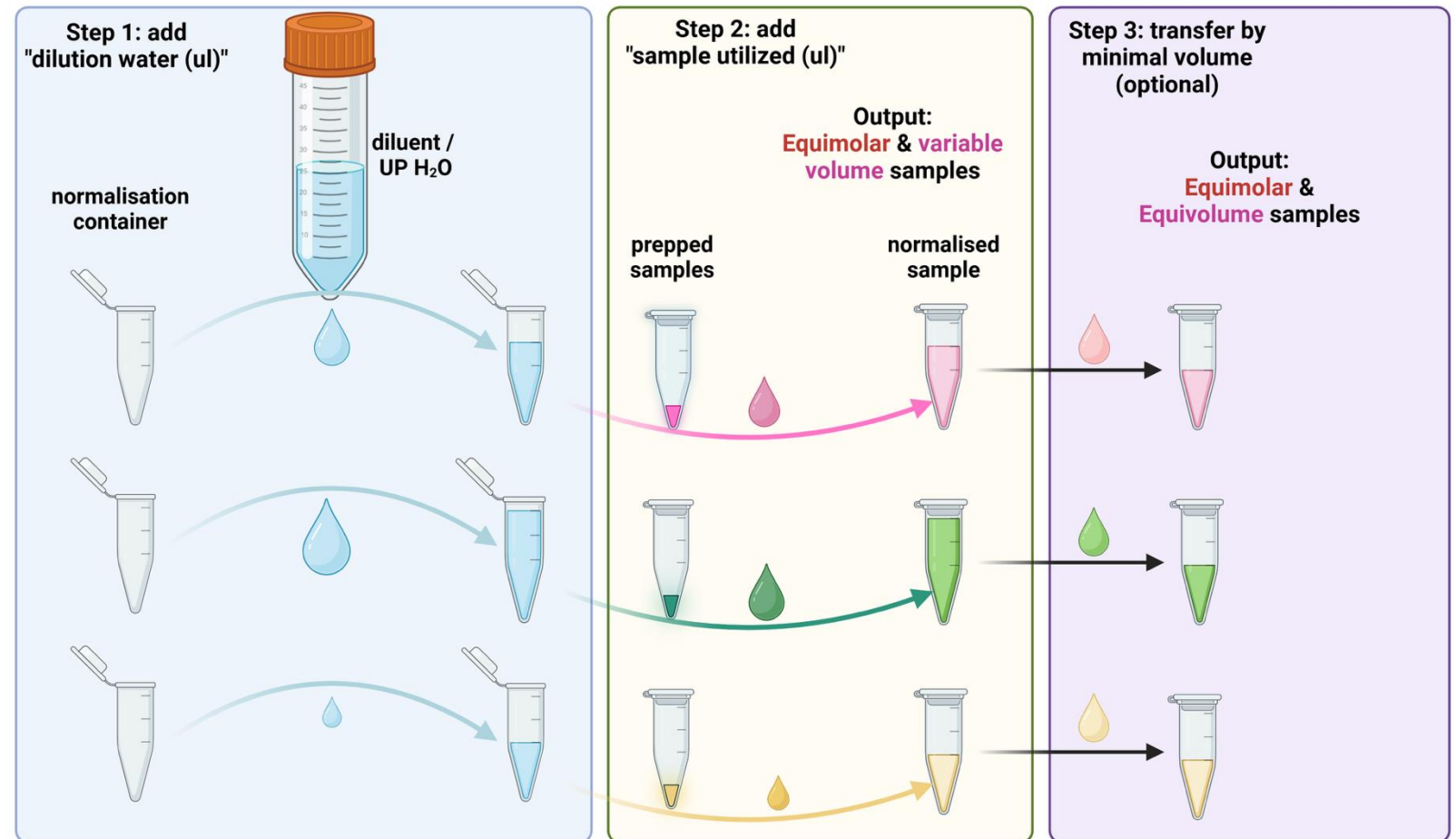
96 samples, 96 concentrations.  
One target for all.

## IN

- › 1–2 plates of samples (up to 192)
- › a design file: one row per sample, with its concentration
- › water in a 60 mL trough

## OUT

- › every sample at the target concentration, each in its own volume
- › a results file: per sample, what was asked, measured and done



# Case study I: Normalisation



How much water? The same sample volume from every well; the water varies

```
sample_uL = sample_volume_uL # the same for all
diluent_uL = sample_uL × (conc / target - 1)
final_uL = sample_uL + diluent_uL # varies
```

From the design file: 10 µL of every sample

| Well | sample<br>ng/µL | target<br>ng/µL | available<br>µL | water<br>µL | final<br>volume µL |
|------|-----------------|-----------------|-----------------|-------------|--------------------|
| A1   | 55              | 10              | 24              | 45          | 55                 |
| A2   | 120             | 10              | 24              | 110         | 120                |
| D1   | 12              | 10              | 24              | 2           | 12                 |

Every well ends at 10 ng/µL, each in its own final volume.

Each sample gets a status

**normalise**

when the usual case  
then the full sample volume, diluted to the target

**sample\_adapted**

when too little sample, or the result would overflow the well  
then a smaller sample volume, still diluted to the target

**too\_dilute**

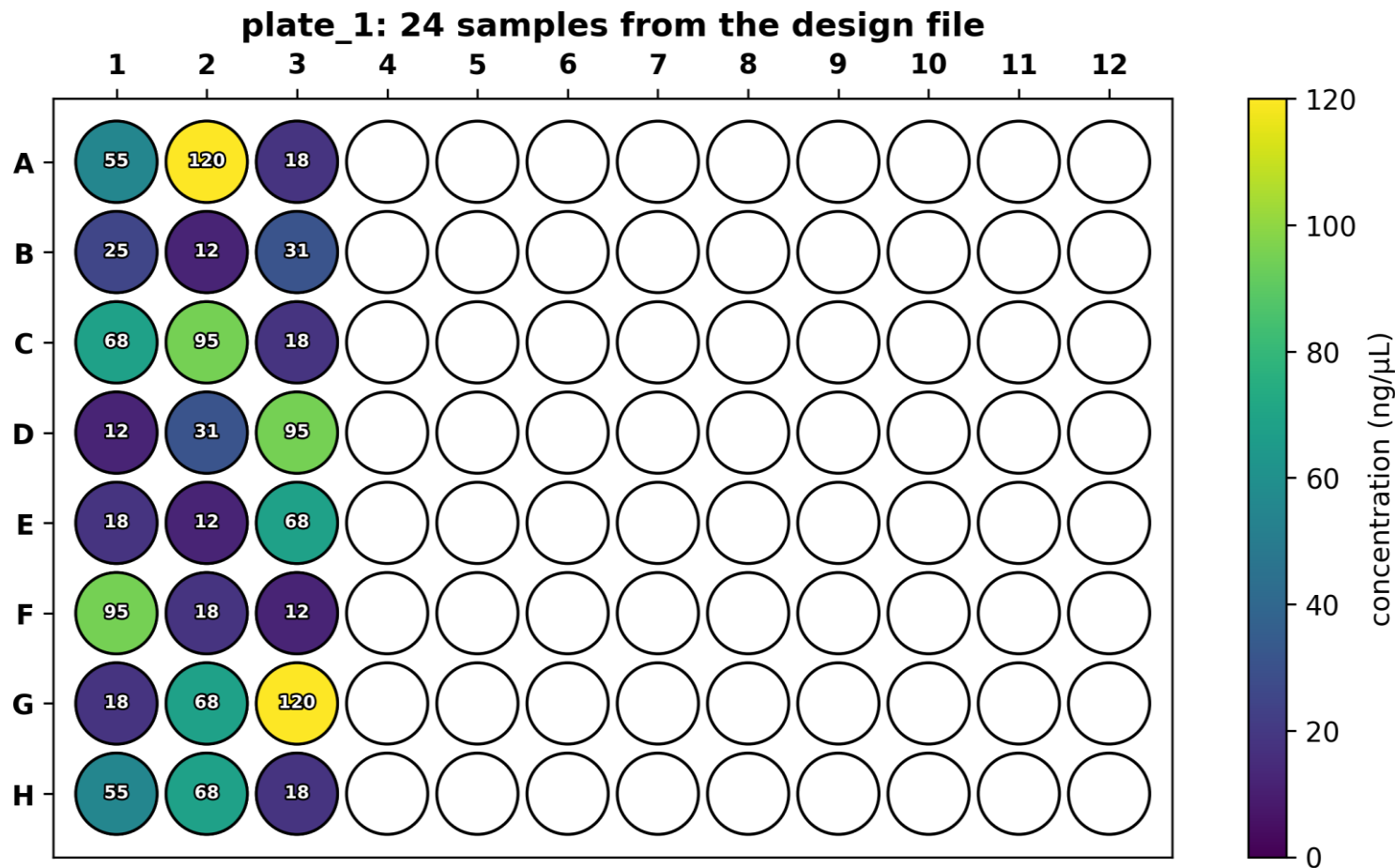
when already below the target  
then no water; flagged in the results file

**Why the Prep's 2 channels, not its 8-channel head?** One piston drives all 8 head tips: one volume. The 2 channels each take their own.

# Case study I: Normalisation



## Notebook 5: the design file, one row per sample



```
plate_id,well_id,...,concentration_ng_uL,  
available sample to dilute (uL)  
plate_1,A1,Sample 1,DNA,55,24  
plate_1,B1,Sample 2,DNA,25,24  
...
```

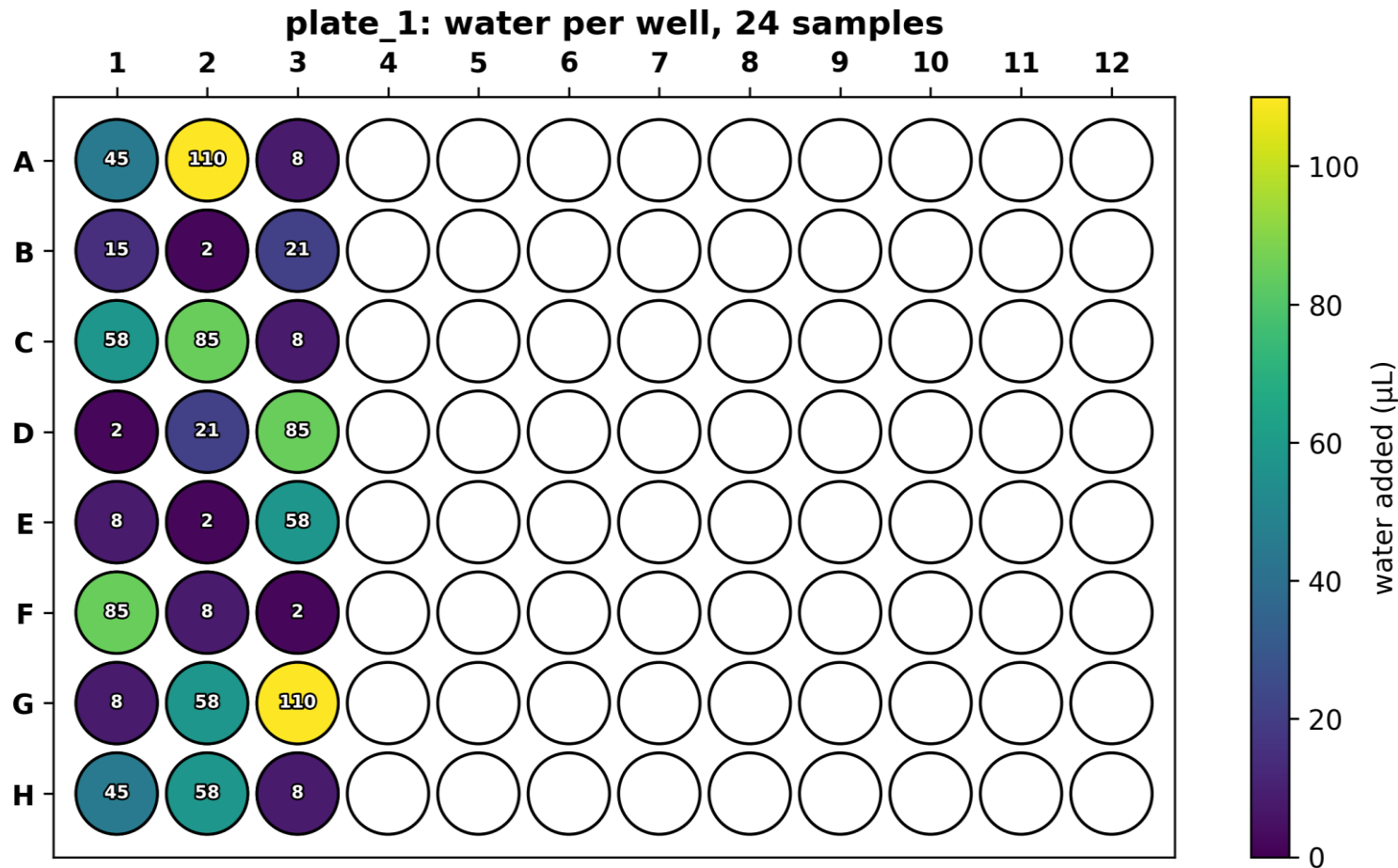
### WHAT THE NOTEBOOK READS

- › 24 samples on plate\_1, columns 1–3
- › 12 to 120 ng/μL, 24 μL available each
- › checked first: the columns, real wells, no well twice
- › target: **10 ng/μL** for every sample

# Case study I: Normalisation



Notebook 5: the water per well, computed from the design file



$$\text{water} = 10 \mu\text{L} \times (\text{conc} / 10 - 1)$$

$$\text{final} = 10 \mu\text{L} + \text{water}$$

## ONE VOLUME PER WELL

- › 10 µL of every sample
- › 2 to 110 µL of water: **24 transfers, each its own volume**
- › final volume 12 to 120 µL; all fit the 200 µL wells
- › ≤ 50 µL: 50 µL tips; above: 300 µL tips
- › each of the Prep's 2 channels takes its own volume

# Case study I: Normalisation



## Notebook 5: Normalisation

- Measure: water trough before start
  - (1) is filled,
  - (2) has enough liquid
- Control:
  - (1) compute variable volumes per pipette
  - (2) compute variable liquid classes per pipette
- Harness:
  - (1) simple CSV input for enabling variable input
  - (2) use AI for input permutation testing



# Lunch Break

12:00 – 13:00 · see you back for multi-device integration on the Hamilton Prep



# Case study II: Hit the Colour

Session 3 13:00 – 14:30 · the Prep measures, models, acts and corrects itself

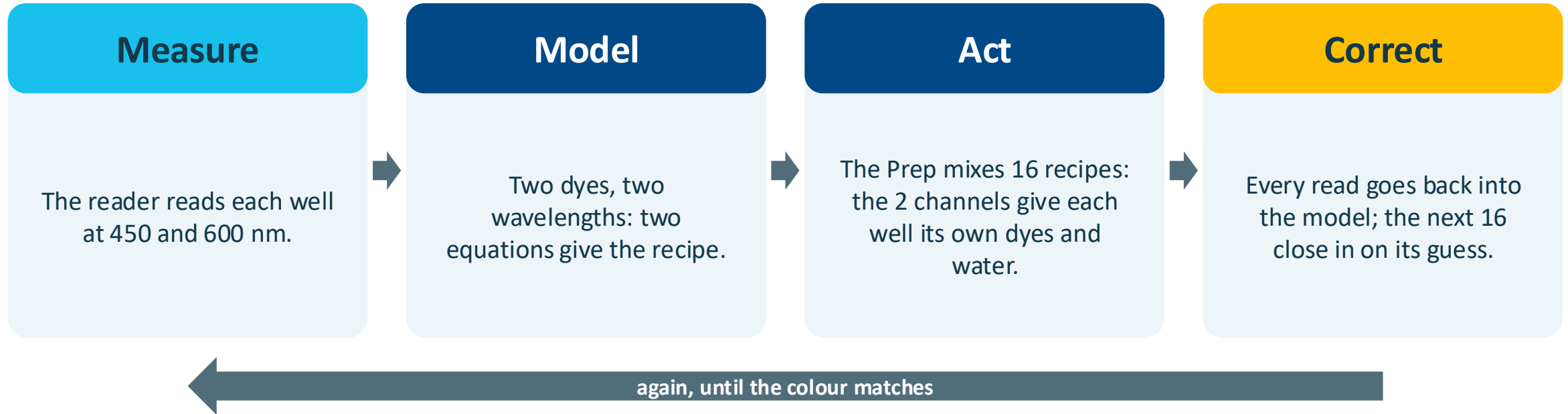


▶ hands-on in a notebook

# The Loop



Industry has formulated colours this way since 1966. Now the Prep does it, closed-loop.



**The measurement decides the next step.** That is self-driving automation, in one script.

Computer colour matching: Allen, J. Opt. Soc. Am. 56:1256 (1966). Colour matching as a minimal self-driving lab: Baird & Sparks, Matter 5:4170 (2022).





# Two Dyes, Two Wavelengths

Each wavelength mostly sees one dye, so two readings give two amounts

| Food dye                                                                                        | 450 nm | 600 nm      |
|-------------------------------------------------------------------------------------------------|--------|-------------|
|  Yellow (E102) | strong | almost none |
|  Blue (E133)   | weak   | strong      |

**Beer–Lambert law:** absorbance rises in a straight line with the amount of dye, and two dyes simply add up.

# per  $\mu\text{L}$  of dye, from the calibration wells

$$A_{450} = y \times Y_{450} + b \times B_{450}$$

$$A_{600} = y \times Y_{600} + b \times B_{600}$$

# Allen's method (1966): solve for the dyes

`y, b = numpy.linalg.solve(M, target)`

y, b:  $\mu\text{L}$  of yellow and blue for one well. M: the calibration

# Case study II: Hit the Colour



The model: two dyes, two wavelengths, two straight lines, refitted after every batch

## 1 One well, two readings



y  $\mu$ L yellow  
b  $\mu$ L blue

**A450**  
mostly yellow

**A600**  
mostly blue

minus a water blank

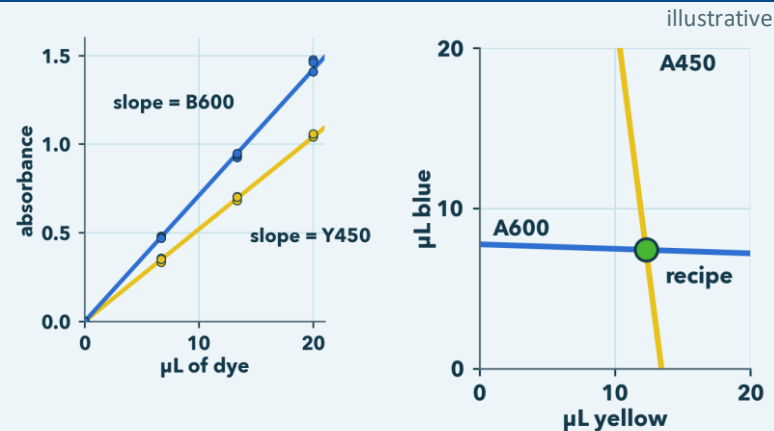
**Beer–Lambert:** a straight line in each dye, and the dyes add up

$$A_{450} = y \cdot Y_{450} + b \cdot B_{450}$$

$$A_{600} = y \cdot Y_{600} + b \cdot B_{600}$$

**Y450 ... B600:** absorbance per  $\mu$ L of each dye = the 2  $\times$  2 model

## 2 Fit the model, solve the recipe



### Fit

all reads so far: more reads, better model

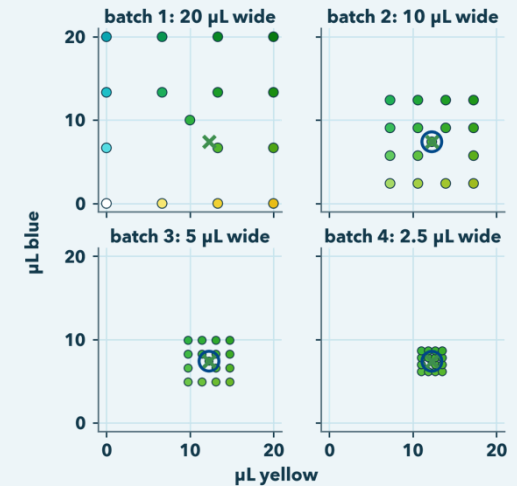
```
lstsq(recipes,  
reads)
```

### Solve (Allen, 1966)

2 readings, 2 unknowns; kept in 0–20  $\mu$ L

```
solve(model,  
target)
```

## 3 The loop closes in



○ the guess    × the target's recipe (illustrative)

**Each batch:** pipette 16, read, refit on all reads, solve = the guess. Next 4  $\times$  4 grid: around the guess, half as wide.

**Stop:** best well within 0.01 of the target (both wavelengths). **Limit:**  $\sim$ 1 % pipetting CV =  $\sim$ 0.01 absorbance at 16  $\mu$ L.

# Case study I: Normalisation

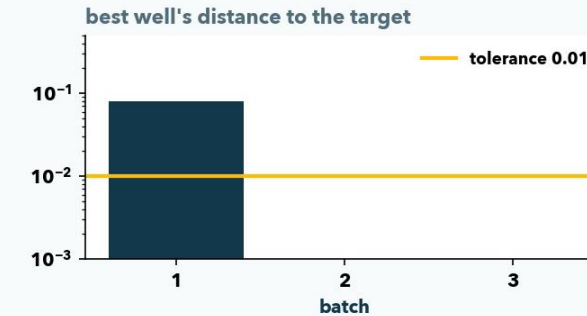
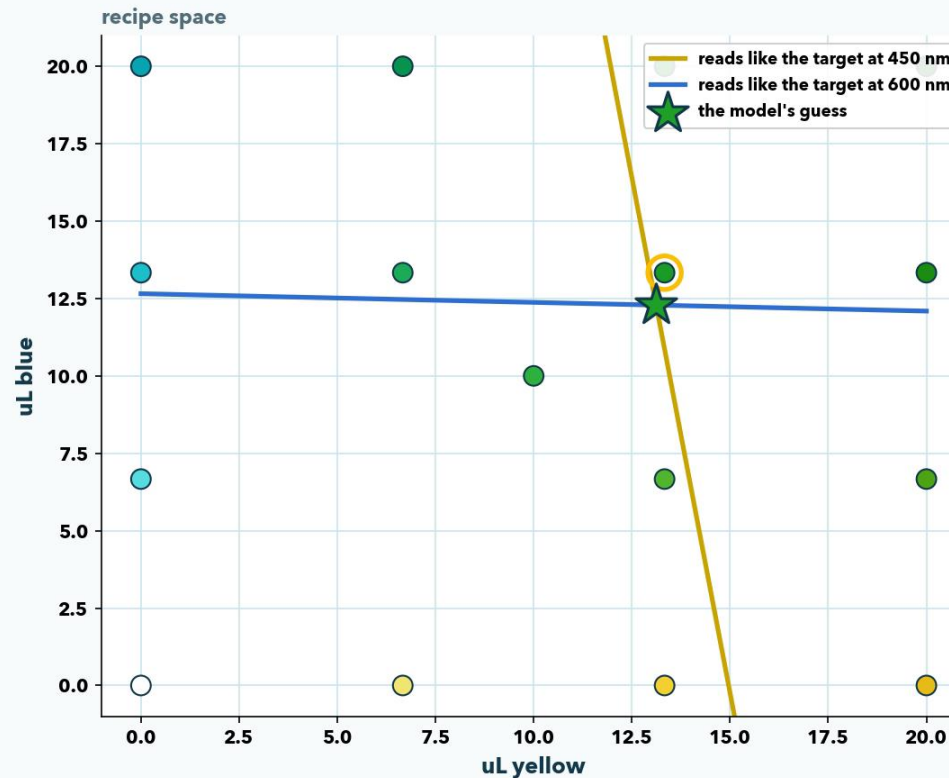


The model at work: learn four numbers, solve for the guess, zoom in, repeat

## How the model finds a recipe

**Solve: where the two lines cross**

On the gold line a recipe reads like the target at 450 nm, on the blue line at 600 nm. Only the crossing does both: the guess.



the model: what 1 uL adds

|             | at 450 nm | at 600 nm |
|-------------|-----------|-----------|
| 1 uL yellow | 0.052     | 0.002     |
| 1 uL blue   | 0.008     | 0.071     |

target



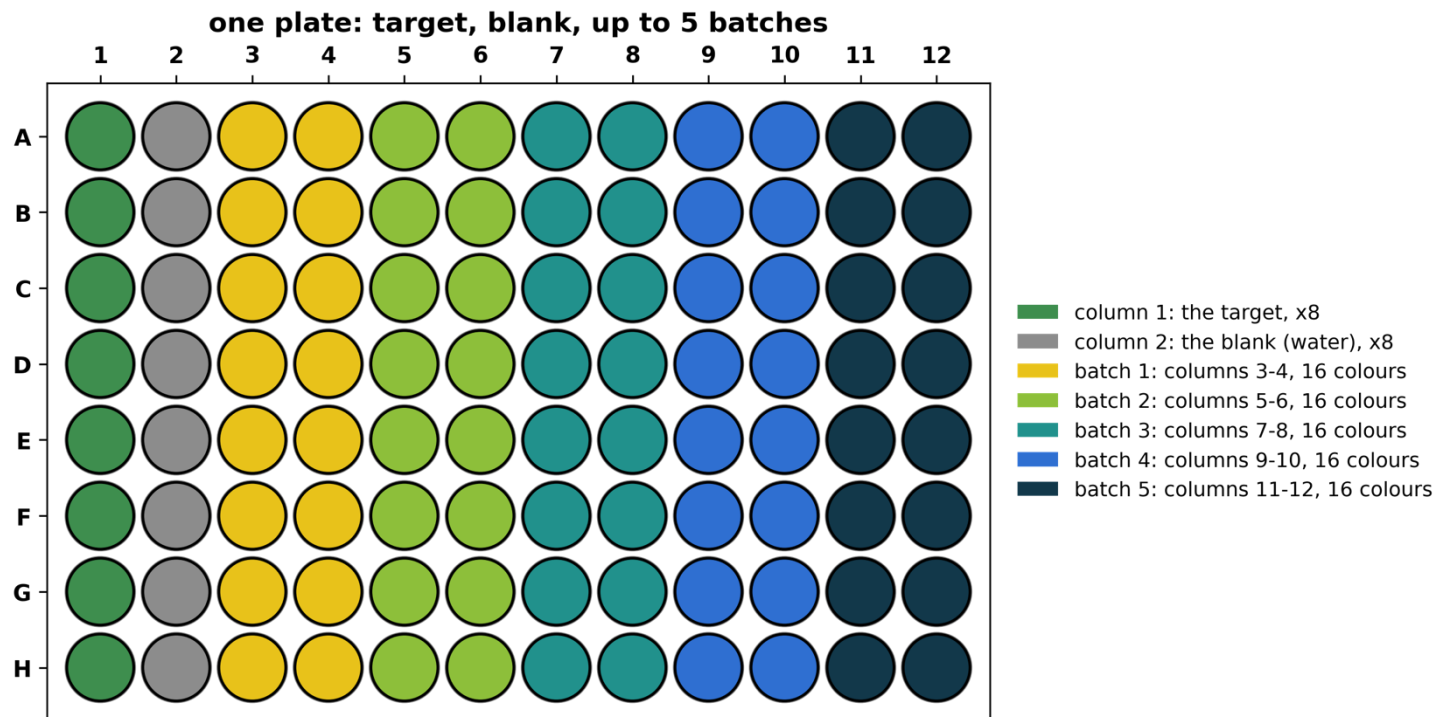
best so far



# One Run on the Prep



One plate: the target, a blank, then batches of 16 colours until one matches



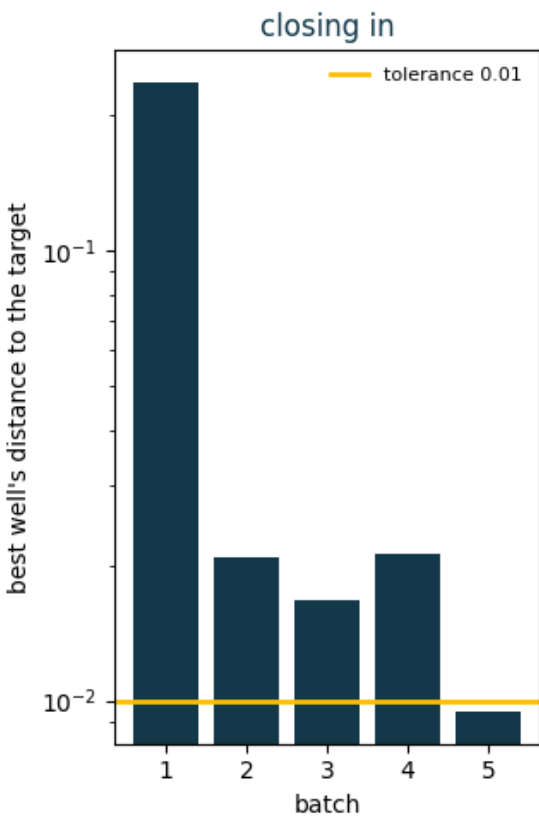
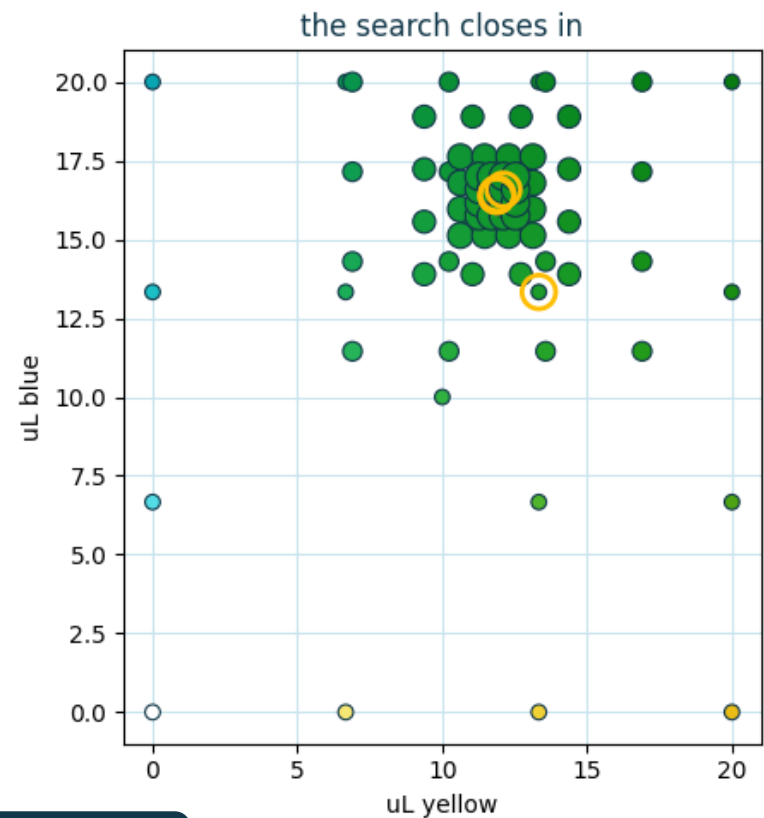
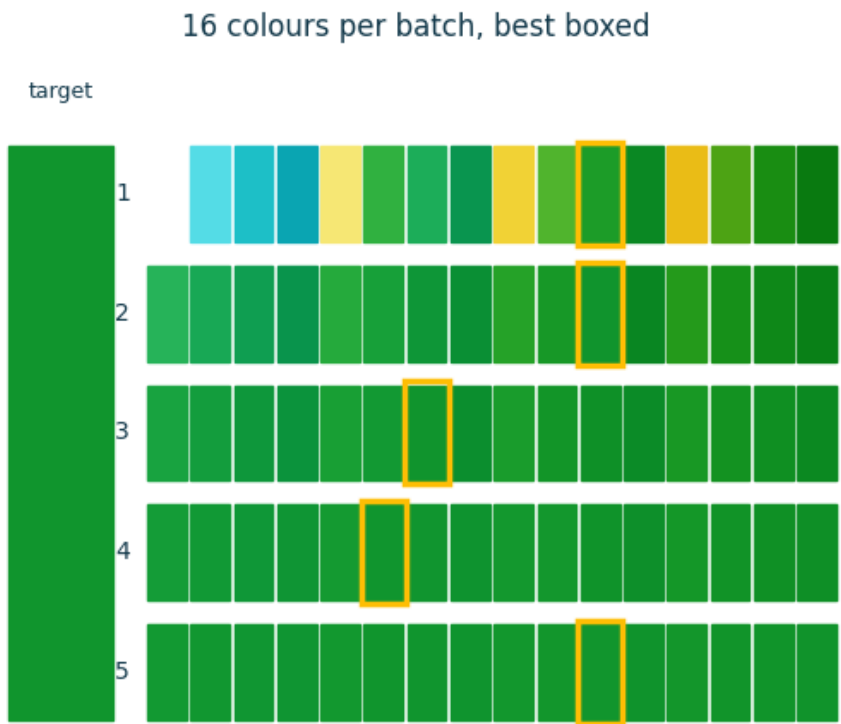
## WHO DOES WHAT

- › **Teaching needle:** measures every trough once
- › **8-channel head:** target, blank, and mixing, a column at a time
- › **2 channels:** yellow, blue, water: each well its own volume
- › **Grippers:** plate, lid, the reader's illumination unit; a full plate into the trash
- › **Reader:** 450 and 600 nm
- › **Python:** fit, solve, the next 16

# The Loop Closes In



A simulated run: every batch aimed at the model's guess, until a well matches



**Thursday in the Tale:** the reader decides, in the same script.

**Not a toy:** two dyes at two wavelengths is how ISO 8655-7 verifies pipetting volumes.

Simulation: seeded target, 1% pipetting CV. Pipetting verification: ISO 8655-7:2022, Annex B (dual-dye ratiometric photometry).

# nb: Hit the Colour

---



**Notebook 6: a closed loop on the Prep, from target to matched colour**

**... head, channels, grippers, reader: one script**



# PLR in the Automation Value Generation Chain

## End-user Delivery

Session 4 15:00 – 16:00 · from crude lysate to a number, in one run on the Prep



▶ hands-on in a notebook



# Happy Automating, everyone!

For more information:

- PyLabRobot forum
- GitHub repository
- (quite active on LinkedIn)